

Java Programming

SOE104 – Lecture 4

Dr. Erick Purwanto



Xi'an Jiaotong-Liverpool University

西交利物浦大学

SOE104 Java Programming

Lecture 4

For Loops, Do-While Loops, Arrays, and Static Methods

Welcome to SOE104 – Lecture 4 !

- Welcome to **Lecture 4** – For & Do-While Loops, Arrays, Static Methods
- **Learning Outcomes:** in this lecture we are continuing on Control Flow
  - For and Do-While
    - using for-loops to solve problems
    - nested for-loops
    - do-while loops
  - Arrays
    - defining and creating arrays
    - array operations
    - multi-dimensional arrays
  - Static Methods
    - introduction to functions/static methods

Part 1: For and Do-While Loops



- In the first part of the lecture we are going to continue discussing loops
  - in particular, `for` loops
  - the other type of loops, the `while` loops have been discussed last week

Review: While Loops



- Write a Java program that takes integer input n and prints n first powers of two starting from $2^0 = 1$

```
public class PowersOfTwo {
    public static void main(String[] args) {

        // code to receive user input to n

        int i = 1, val = 1;
        while (i <= n) {
            System.out.println(val);
            val = val * 2;
            i = i + 1;
        }
    }
}
```

In-Class Quiz 1: While Loop Bugs



- The following code should print the first n powers of two.

  Identify the bug and **explain** how to fix it.

```
public class PowersOfTwo {
    public static void main(String[] args) {

        // code to receive user input to n

        int i = 0, val = 1;
        while (i <= n) {
            System.out.println(val);
            val = val * 2;
            i = i + 1;
        }
    }
}
```

Reminder, errors are:

- compile error
- runtime error
- infinite loop
- off-by-one bug

Powers of Two



- Write a Java program that takes integer input n and prints n first powers of two starting from $2^0 = 1$

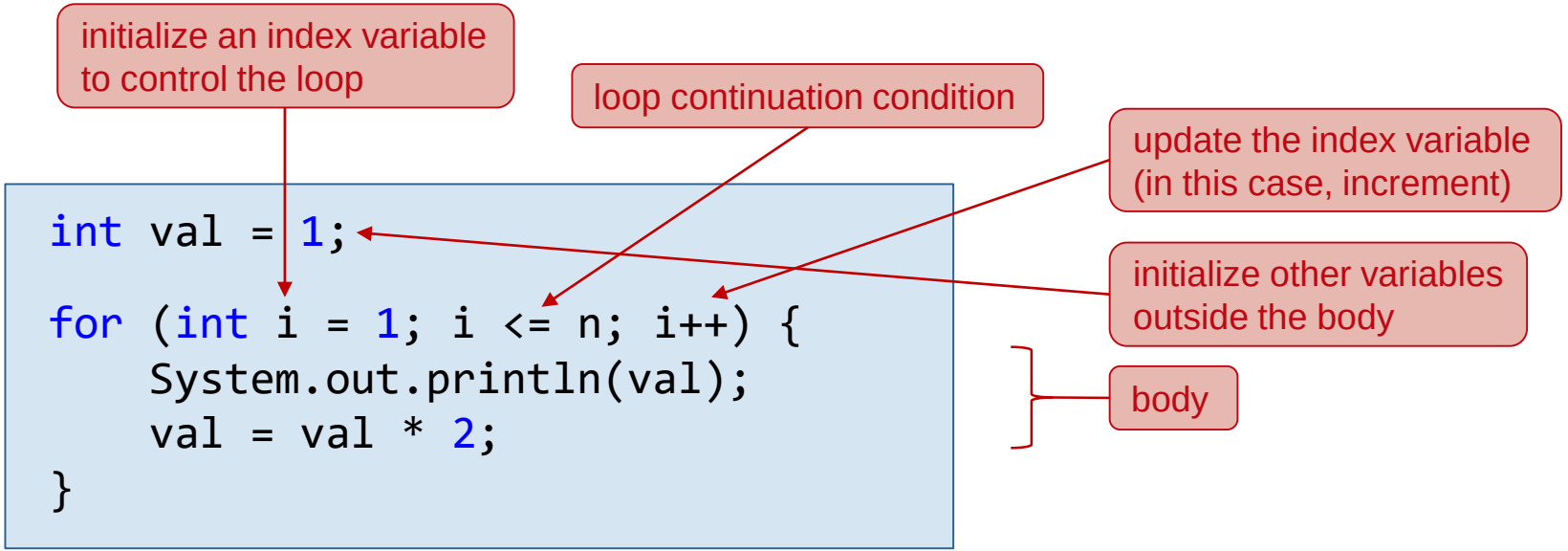
```
public class PowersOfTwo {
    public static void main(String[] args) {

        // code to receive user input to n

        int i = 1, val = 1;
        while (i <= n) {
            System.out.println(val);
            val = val * 2;
            i = i + 1;
        }
    }
}
```

for loops

- for loop repeats statements in its body using **three** statements:
  - **initialize** an index variable (e.g. `i`, `j`, `k`) to some value
  - **test** a continuation condition involving the index, continue looping if it's still true
  - use the last statement to **update** the index variable



In-Class Quiz 2: For Loop

- The following `for` loops iterate exactly 10 times:

```
for (int i = 0; i < 10; i++)
```

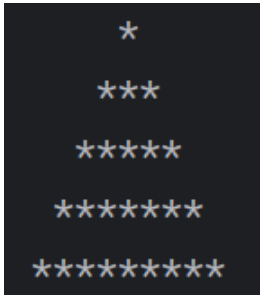
```
for (int i = 1; i <= 10; i++)
```

  Write *another* Java `for` loop that executes exactly 10 times

Nested For Loops

- We can use for loops inside a for loop
  - for example, we use three index variables that depend on the other, to print a pyramid pattern:

```
for (int i = 1; i <= 5; i++) {
    for (int j = i; j < 5; j++) {
       System.out.print(" ");
    }
    for (int k = 1; k <= (2 * i - 1); k++) {
       System.out.print("*");
    }
    System.out.println();
}
```



Problem Solving 1: Harmonic Numbers

QUIZ
Lecture Example 1

Harmonic numbers are a sequence of numbers that arises in various fields of mathematics.
The n-th harmonic number, denoted as H_n, is defined as the sum of the reciprocals of the first n natural numbers:

$$H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} = \sum_{k=1}^{n} \frac{1}{k}$$

Write a Java program *HarmonicNumber* that on input a positive integer n, prints the n -th harmonic number.

Test case 1:
Input:
2
Output:
1.5

Test case 2:
Input:
10
Output:
2.9289682539682538

Test and Debug in your own laptop

Harmonic numbers are a sequence of numbers that arises in various fields of mathematics.
The n-th harmonic number, denoted as H_n, is defined as the sum of the reciprocals of the first n natural numbers:

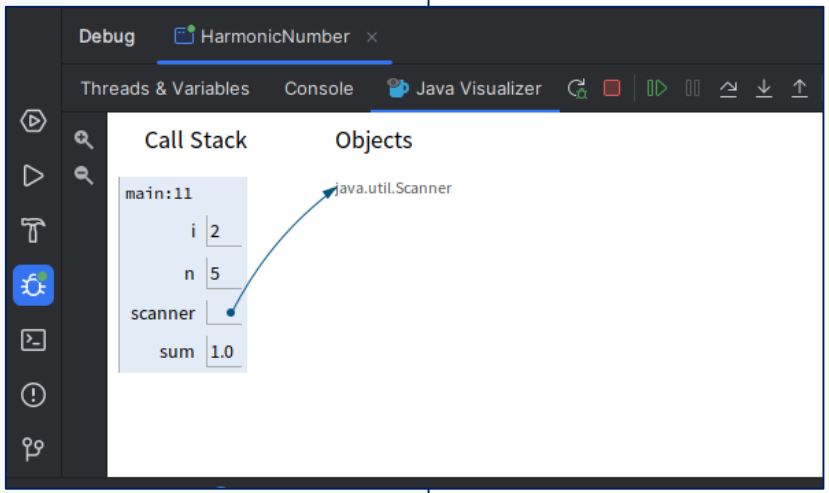
$$H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} = \sum_{k=1}^{n} \frac{1}{k}$$

Write a Java program *HarmonicNumber* that on input a positive integer n, prints the n -th harmonic number.

Test case 1:
Input:
2
Output:
1.5

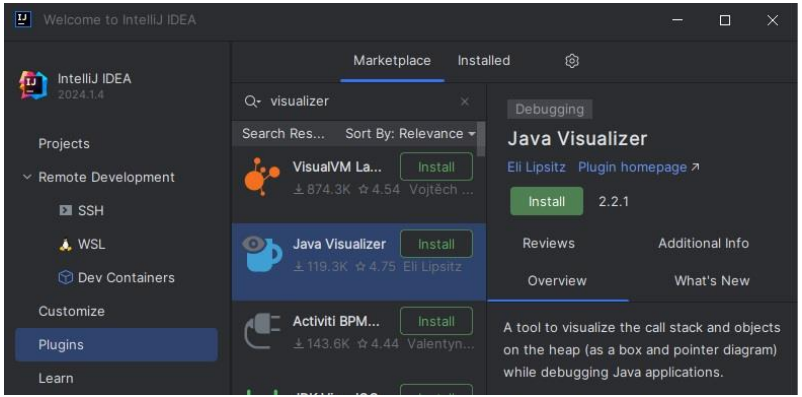
Test case 2:
Input:
10
Output:
2.9289682539682538

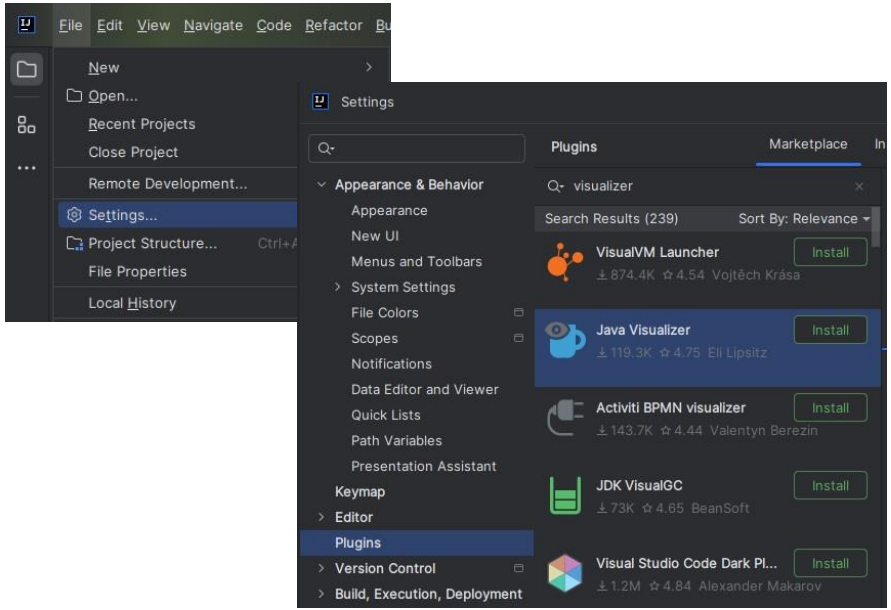
we can use **debugger** and **visualizer** to *test* your code!



Java Visualizer

- Open IntelliJ IDEA
  - in Project Welcome Screen or Main Menu - Settings
  - search "visualizer" and install **Java Visualizer**





Testing

- You are given one or more test cases
  - there are given test cases and hidden test cases
  - your code is **correct** *if and only if* it passes **all** test cases

$$H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} = \sum_{k=1}^{n} \frac{1}{k}$$

Test case 1:
Input:
2
Output:
1.5

Test case 2:
Input:
10
Output:
2.9289682539682538

- Add **more** test cases!
  - create and compute ***your own*** test cases

Buggy Code



- You are given the following formula:

$$H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} = \sum_{k=1}^{n} \frac{1}{k}$$

- Suppose you wrote the following (buggy) code:

```
double sum = 0.0;
for (int i = 1; i <= n; i++) {
    sum += 1 / i;
}

System.out.println(sum);
```

  - and on input n = 5, the output is wrong:

```
5
1.0
```

how can we find the bugs?

Debugging Attempt

- How can **find** and **test** the lines that may contain bugs?
- One way is to `println` the intermediate results, and compare with the correct values computed manually

```
double sum = 0.0;
for (int i = 1; i <= n; i++) {
    sum += 1 / i;
}

System.out.println(sum);
```

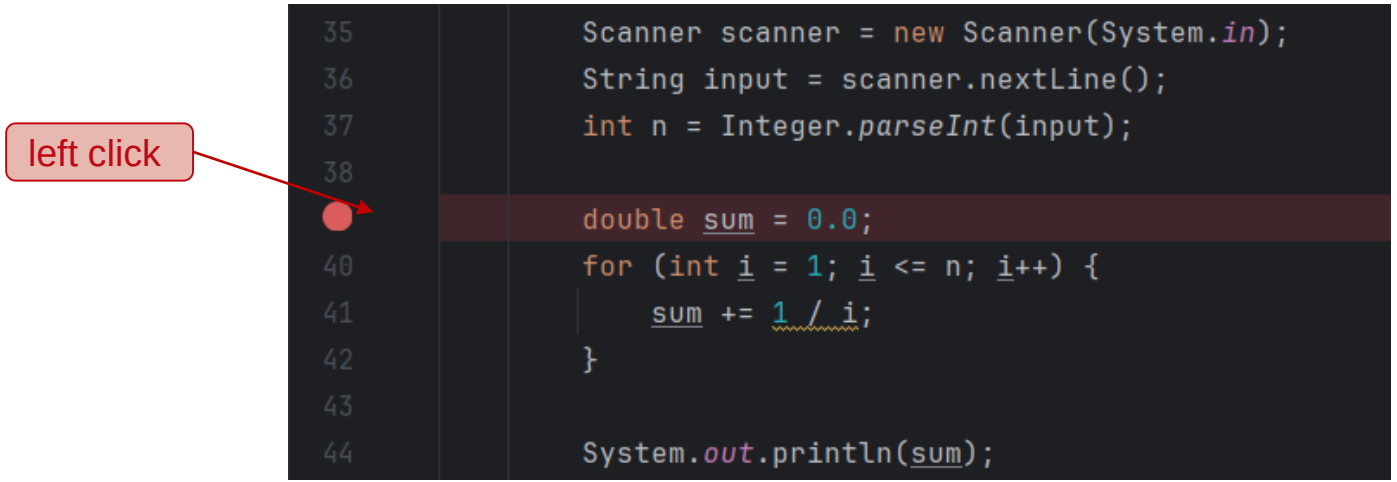
add println(sum) here

add other println here

- need to modify the code
- may forget to remove later
-> is there a better way?

Using Debugger

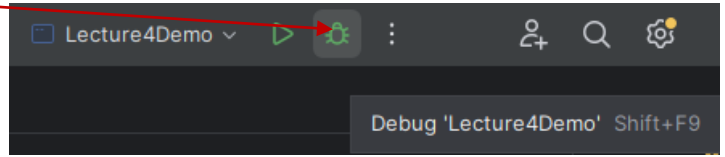
- Set a **breakpoint** by clicking the area close to the line numbers
  - pick the line where you want the program execution to stop



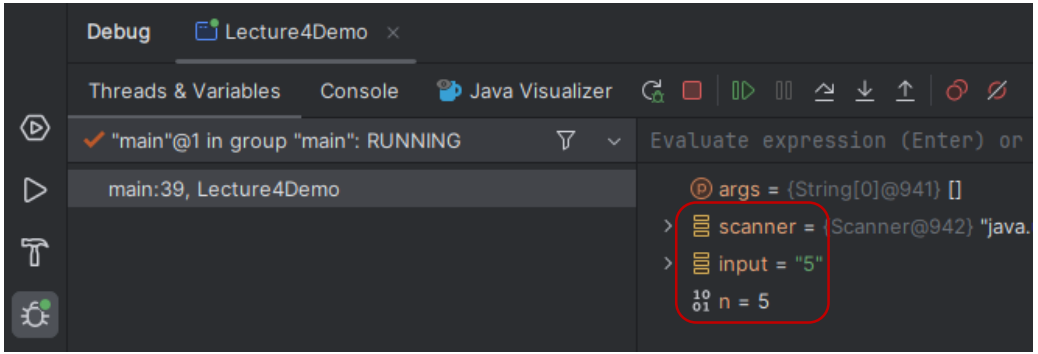
  - after that, the variables will be observable line-by-line

Debugger View

- Click the **Green Bug** icon

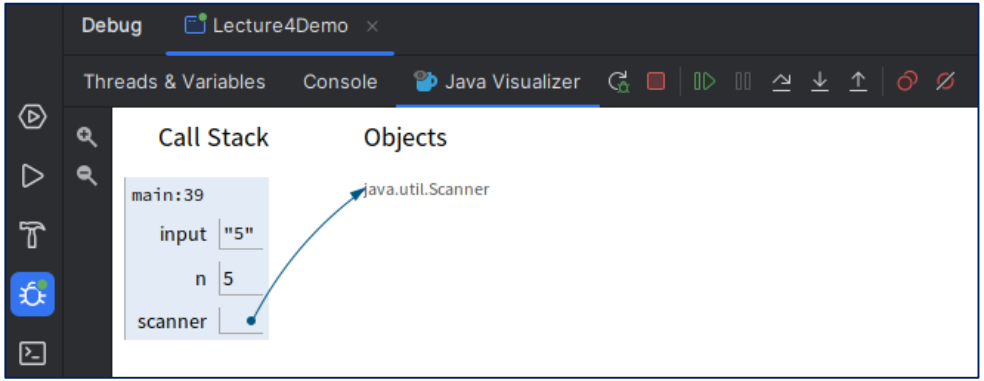


- You can see the current state of the program (the variables n, sum, i) ***before*** that line is executed (before variable sum is initialized)



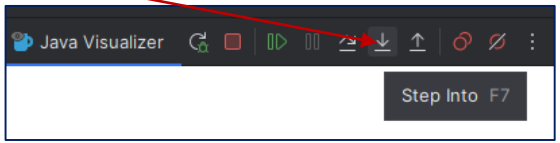
Java Visualizer

- Alternatively, we can use **Java Visualizer** to illustrate the state of the program
  - currently, there are only boxes to show all the variables, but the illustration will become more helpful later as we code more complicated program

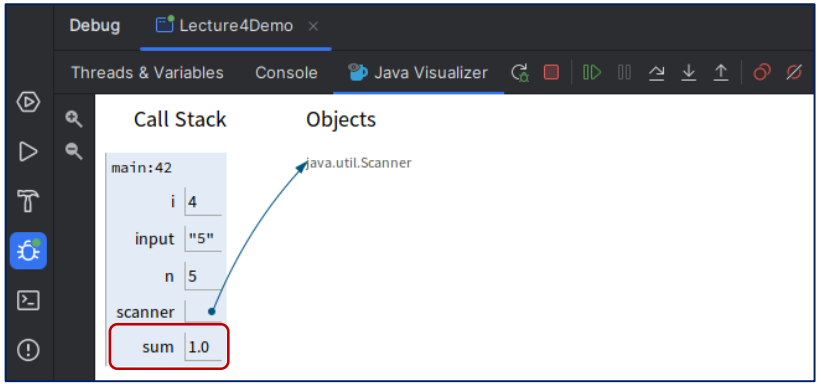


Step Into

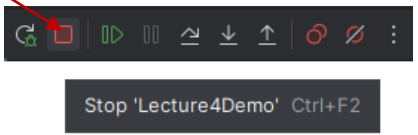
- Click **Step Into** to execute a line of the program



- We learn the value of sum becomes 1.0 and does not change as we iterate



- Stop debugging by clicking **Stop**, and fix the code



could you describe the bug?
how to fix it?

Do-While Loops

- A **do-while loop** allows code to be executed *at least once*, and then *repeatedly*, as long as a given condition evaluates to `true`
  - in the example below, we want to keep rolling dice *until* we get a six

```
public class DiceRoll {
   public static void main(String[] args) {

      int roll;

      do {
           roll = (int)(Math.random() * 6) + 1;
           System.out.println("Rolled: " + roll);
      } while (roll != 6);

      System.out.println("You got a six!");
   }
}
```

we need to ***at least roll the dice once***, so we need a do-while loop

generate random 1-6

keep rolling until you get a six

In-Class Quiz 3: Do-While Loops

- What does the following code calculate for a non-negative integer n?

```
public class MysteryDoWhileLoop {
    public static void main(String[] args) {

        // code to receive user input to n

        int digits = 0;

        do {

            digits++;
            n = n / 10;

        } while (n > 0);
    }
}
```

Part 2: Arrays

- In Part 2, we discuss **arrays**, which are simple ordered collections of data
  - we can have an array of any given data type (primitives, objects)
  - in Java, we ***cannot*** mix the elements, e.g. some type double, others type int

```
int n = 5;
double[] a;
a = new double[n];
for (int i = 0; i < n; i++) {
    a[i] = 0.0;
}

double[] b = new double[]{1.0, 2.5, 3.6};

int[] c = {4, 5, 6, 7, 8};
```

declare the array

create the array

the elements are indexed from `0` to `n-1`

initialize all elements to `0.0`

alternative array initializations

  - **default initialization**: Java initializes all entries in arrays of numbers to `0` and in arrays of Booleans to `false`

visualize it!

Array Operations

- Arrays are **fixed-length** sequences of another type

- To declare an array variable and construct an array value to assign to it:
  - `int[] a = new int[100];`
  - it includes all possible int array values, but once an array is created, we can **never** change its length

- Operations on array include:
  - indexing `a[2]`
  - assignment `a[2] = 5`
  - length `a.length`

note that this is different syntax from `String.length()`
`a.length` is **not** a method call, so don't put parentheses after it

Printing and Enhanced For Loops

- Arrays are a **reference** data type
  - array variable stores the address of the first element in the memory

```
String[] months = {
    "", "Jan", "Feb", "Mar", "Apr", "May", "Jun",
    "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"
};
System.out.println(months);

for (int i = 1; i < months.length; i++) {
    System.out.println(months[i]);
}

for (String s : months) {
    System.out.println(s);
}
```

printing the address

printing the contents

enhanced for loops to iterate over *all* elements in the array

visualize it!

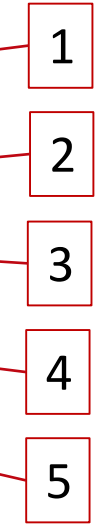
In-Class Quiz 4: Array Example

- The code attempts to **find the maximum value in a non-empty array**
- Explain how it may produce an ***incorrect*** result and suggest a ***correction***!

```
double[] arr = {2.5, 100.01, 99.99};

double max = 0.0;

for (int i = 0; i < arr.length; i++) {
    if (arr[i] > max) {
        max = arr[i];
    }
}
System.out.println(max);
```



Two-dimensional Arrays

- To represent a **matrix**, or a **table** of numbers organized in `m` rows and `n` columns, we use a **two-dimensional array** in Java
  - To refer to the element in row `i` and column `j` of a two-dimensional array `a[][]`, we use the notation `a[i][j]`

```
int m = 3, n = 5;
double[][] a = new double[m][n];

for (int i = 0; i < m; i++)
    for (int j = 0; j < n; j++)
        a[i][j] = i + j + 1.0;

for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++)
         System.out.print(a[i][j] + " ");
    System.out.println();
}
```

Output:

```
1.0 2.0 3.0 4.0 5.0
2.0 3.0 4.0 5.0 6.0
3.0 4.0 5.0 6.0 7.0
```

Multidimensional Arrays

- The same notation extends to arrays that have any number of dimensions
  - for instance, we can declare and initialize a three-dimensional array with the code

```
int n = 5;
double[][][] a = new double[n][n][n];
```

  - and then refer to an entry with code `a[i][j][k]`

Multidimensional Array Example

- Complete the following program to generate a Pascal's Triangle

```
int[][] triangle = new int[n][n];

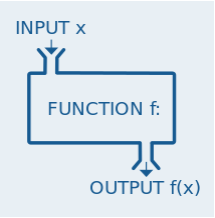
for (int i = 0; i < n; i++)
    for (int j = 0; j <= i; j++)
        if (j == 0)
            triangle[i][j] = 1;
        else
            triangle[i][j] = ... ;
```

?

```
5
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

- `triangle[i - 1][j - 1] + triangle[i][j]`
- `triangle[i - 1][j + 1] + triangle[i][j - 1]`
- `triangle[i - 1][j - 1] + triangle[i - 1][j]`
- `triangle[i - 1][j + 1] + triangle[i - 1][j - 1]`

Part 3: Static Methods 1



- Functions take an **input** and produce an **output**
  - in Java, functions are implemented as **static methods**

- We have used static methods/functions before!
  - for example, in Lecture 2 we used **`Math.abs()`** function
  - when we write **`y = Math.abs(x)`** in a program, the effect is that the function is replaced by the output or *return value* that is produced by Java's **`Math.abs(x)`** method when we passed the number x as an **argument**

Find Maximum Number

- Example: Write a Java program that find the **maximum** of two integers
- You may write the following program in `main`:

```
public class Lecture4Demo {
    public static void main(String[] args) {
        int x = 2, y = 5;
        if (x > y)
            System.out.println(x);
        else
            System.out.println(y);
        x = 10;  y = 8;
        if (x > y)
            System.out.println(x);
        else
            System.out.println(y);
    }
}
```

you test your code on the case where the second number is larger

then, you test your code on the case where the first number is larger

notice the repeated codes!

Static Methods

- We want to avoid **repeated** codes, and want to **organize** code as independent parts to share and **reuse** code to build bigger programs

```
public class Lecture4Demo {

    public static int larger(int x, int y) {
        if (x > y) {
            return x;
        }
        return y;
    }

    public static void main(String[] args) {
        System.out.println(larger(2, 5));
        System.out.println(larger(10, 8));
    }
}
```

instead, write your program **once** inside a function/static method

now you can **call** your function in main **multiple** times!

Static Method Definition

- A static method consists of a **header** and a **body**
  - the body is enclosed in curly braces

```
public class Lecture4Demo {

    public static int larger(int x, int y) {
        if (x > y) {
            return x;
        }
        return y;
    }

    public static void main(String[] args) {
        System.out.println(larger(2, 5));
        System.out.println(larger(10, 8));
    }
}
```

give a descriptive **name** to a static method

specify the **parameter(s)** -> *input* to the function

specify the **type** of each *parameter* variable

specify the **type** of the *return* / *output* value

return the output value, do ***not*** println it
-> **return statement(s)**

Static Method Call and Trace

- A static method/function call is method **name** followed by its **arguments**
  - separated by commas and enclosed in parenthesis

```
public class Lecture4Demo {

    public static int larger(int x, int y) {
        if (x > y) {
            return x;
        }
        return y;
    }

    public static void main(String[] args) {
        System.out.println(larger(2, 5));
        System.out.println(larger(10, 8) + 5);
    }
}
```

function call to be printed

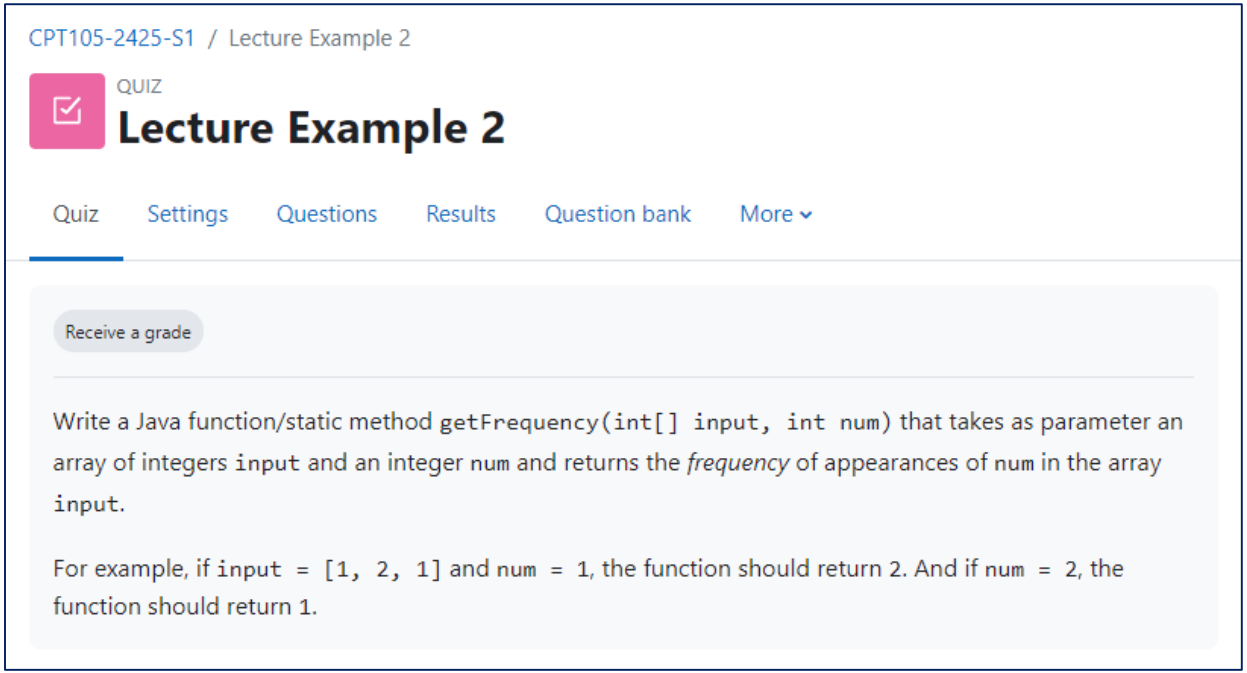
arguments must match the parameter type(s)

a function call is an *expression*, can be part of more complicated expressions

visualize using **Java Visualizer**!

Problem Solving 2

- Solve this problem and write a function that takes an array and an integer, and returns an integer



CPT105-2425-S1 / Lecture Example 2

QUIZ

Lecture Example 2

Quiz | Settings | Questions | Results | Question bank | More

Receive a grade

Write a Java function/static method `getFrequency(int[] input, int num)` that takes as parameter an array of integers `input` and an integer `num` and returns the *frequency* of appearances of `num` in the array `input`.

For example, if `input = [1, 2, 1]` and `num = 1`, the function should return 2. And if `num = 2`, the function should return 1.

Test-driven Development

- Create test cases first, before completing your method
  - Two test cases are given. Suggest more test cases!

Write a Java function/static method `getFrequency(int[] input, int num)` that takes as parameter an array of integers `input` and an integer `num` and returns the *frequency* of appearances of `num` in the array `input`.

For example, if `input = [1, 2, 1]` and `num = 1`, the function should return 2. And if `num = 2`, the function should return 1.

For example:

Test	Result
`int[] input = {1, 2, 1};` `System.out.println(getFrequency(input, 1));`	2
`int[] input = {1, 2, 1};` `System.out.println(getFrequency(input, 2));`	1

Testing First

- Create **GetFrequencyTest** class using the previous steps in Slide 25

Lecture4Demo.java | GetFrequency.java

```java
public class GetFrequency {



    public static int getFrequency(int[] input, int num) {



        return 0;
    }

    public static void main(String[] args) {
        int[] arr1 = {1, 2, 1};
        int num1 = 1;
        int num2 = 2;
        System.out.println(getFrequency(arr1, num1));
        System.out.println(getFrequency(arr1, num2));
    }

}
```

keep the function empty first

Solve the Problem

- After you have passed enough test cases in your own computer, submit to Learning Mall

	Test	Expected	Got	
✔	`int[] input = {1, 2, 1};` `System.out.println(getFrequency(input, 1));`	2	2	✔
✔	`int[] input = {1, 2, 1};` `System.out.println(getFrequency(input, 2));`	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 100.00/100.00.

the lecturer will now solve the problem for you...

In-Class Quiz 5: Static Methods

- What does `myFunction(10)` return?
- Explain what the static method calculates!

```
public static int myFunction(int n) {
    int count = 0;
    for (int i = 1; i < n; i++) {
        if (i * i < n) {
            count++;
        }
    }
    return count;
}
```

Thank you for your attention !

- **Conclusion:** in this lecture, you have learned:
  - to leverage for-loops to solve problems
  - to use nested for-loops
  - to utilize do-while loops when need to execute code at least once
  - to store and manipulate data using arrays
  - to use enhanced for-loops to iterate over all entries in an array
  - to solve problems using arrays and multidimensional arrays
  - to create functions / static methods
  - to solve problems modularly using methods

- Please continue to **Online Exercises** and **Lab 4**